

# Loom

## A Contract-Based Language for Human-Scale Formal Proofs

Mathematical intention, bounded reconstruction,  
and independently checked evidence

---

**A design study grounded in ProveIt and Leant**

Prepared for Vladimir Reshetnikov

September 5, 2026

**Central proposal.** Authors should write the choices that make a proof intelligible: its invariant, witness, decomposition, transformation, and decisive lemma. A compiler should reconstruct routine consequences under explicit contracts, and a separate checker should verify the resulting proof of a frozen formal statement. Concision belongs in the document; rigor belongs in the evidence; the correspondence between them must remain inspectable.

Research proposal, not a claim of an implemented language. The mathematical examples are analyzed from pinned repository sources. No Lean build, Leant test suite, or prototype compiler was executed for this study.

## Abstract

A long formal proof often interleaves three different things: a mathematical argument, the obligations that make that argument valid, and instructions for persuading a particular elaborator to recognize it. Removing the second would weaken the mathematics; removing the third from the author’s working document need not. This article proposes Loom, a controlled, extensible proof language layered over Lean. Its unit of expression is a *contracted mathematical step*: a claim or transformation whose meaning, admissible assumptions, and reconstruction obligations are explicit, although its low-level proof is normally hidden.

The design is grounded in three inspected ProveIt modules concerning iterated derivatives, Abel polynomials, and affine-difference orbits, and in Leant’s term-synthesis and verification architecture. Worked rewrites show how neighborhood reasoning, rational-algebra transport, and finite-set bijections can be compressed without strengthening hypotheses or discarding side conditions. A scoped obligation calculus describes how these steps compose. A separate account of statement approval, witness selection, proof replay, and axiom policy distinguishes kernel validity from fidelity to the author’s intention. The article also specifies a practical Leant integration, a staged implementation, and an evaluation protocol that compares against well-refactored Lean rather than an artificially verbose baseline. The intended contribution is not unrestricted natural-language theorem proving, but an auditable interface between mathematical explanation and proof-producing automation.

## Contents

|           |                                                                    |           |
|-----------|--------------------------------------------------------------------|-----------|
| <b>1</b>  | <b>The problem is not that rigor requires verbosity</b>            | <b>2</b>  |
| <b>2</b>  | <b>Precedents and the proposed contribution</b>                    | <b>3</b>  |
| <b>3</b>  | <b>A language of contracted mathematical steps</b>                 | <b>4</b>  |
| <b>4</b>  | <b>Case study I: differentiating an induction identity</b>         | <b>5</b>  |
| <b>5</b>  | <b>Case study II: Lagrange inversion without cast choreography</b> | <b>7</b>  |
| <b>6</b>  | <b>Case study III: a finite sum over the Boolean cube</b>          | <b>10</b> |
| <b>7</b>  | <b>What the language should infer, and what it should expose</b>   | <b>12</b> |
| <b>8</b>  | <b>A scoped obligation calculus</b>                                | <b>14</b> |
| <b>9</b>  | <b>Making Leant a reconstruction service</b>                       | <b>16</b> |
| <b>10</b> | <b>Proof artifacts, reproducibility, and the trust boundary</b>    | <b>19</b> |
| <b>11</b> | <b>The authoring experience</b>                                    | <b>21</b> |
| <b>12</b> | <b>An implementation path that does not require a new prover</b>   | <b>22</b> |
| <b>13</b> | <b>How to evaluate whether the language is actually better</b>     | <b>23</b> |
| <b>14</b> | <b>Limits, tradeoffs, and the central recommendation</b>           | <b>25</b> |
| <b>A</b>  | <b>A minimal syntax sketch</b>                                     | <b>26</b> |
| <b>B</b>  | <b>A recipe contract and its replay record</b>                     | <b>27</b> |
| <b>C</b>  | <b>Source map and boundaries of this study</b>                     | <b>28</b> |

# 1 The problem is not that rigor requires verbosity

Consider an argument a mathematician might write in a single sentence: “Induct on  $n$ ; differentiate the induction identity on the open set, use the chain rule, and substitute the recurrence.” The sentence contains an actual strategy. It selects induction rather than, say, a generating function; it invokes a local differentiability argument; it identifies the relation needed at the end. Its brevity comes from delegating familiar work to the reader.

There are two very different ways to formalize this sentence. One is to expand all of that work into an author-maintained tactic script. Another is to give the sentence a precise interpretation that generates proof obligations, and to ask a proof-producing system to discharge those obligations. The latter is the direction proposed here. It is not a permission to accept an unjustified sentence. It is a way to avoid making every author reproduce its implementation.

A useful distinction is between *mathematical commitments* and *reconstructible detail*. The choice of induction variable, a delicate domain restriction, or a nonobvious substitution is usually a commitment. The ordering of arguments to a theorem, a routine coercion through an algebra homomorphism, or the lambda term converting pointwise information to a local equality is often reconstructible. The division is contextual: a choice routine in one library may be the central idea of a new theorem. Consequently, the language must permit the same step to be written at several levels of detail.

Lean already has implicit arguments, type-class inference, rewriting, calculation blocks, and powerful proof-producing automation. Its current tactic reference includes search and suggestion facilities as well as specialized decision procedures [8, 9]. A proposal that merely renames `have`, `calc`, and `aesop` in English would offer little. The objective is a different *division of responsibility*: the human maintains the explanatory argument, while the system maintains a separately inspectable reconstruction of its routine steps.

Nor is the shortest script necessarily the best proof document. A one-line call to a general search procedure may be less informative than a ten-line proof that exposes an invariant. Conversely, a long script can have excellent structure and deliberately document a subtle argument. The optimization target is therefore not character count alone. It is the amount of unnecessary implementation knowledge required to write, read, modify, and trust a proof.

## 1.1 Scope and evidential status

The source review uses fixed revisions rather than moving branch names:

| Repository | Inspected revision                       |
|------------|------------------------------------------|
| ProveIt    | 24ce8bd743eaab64a91ce90725ea00f498d319d2 |
| Leant      | 3a40904be8a410d832d9ab6900b3d3e7b425eccb |

The ProveIt sample consists of three modules under `Analysis/FabiusFunction/Lean/FabiusFunction/`: `AutonomousIteratedDeriv.lean`, `AbelPolynomialSeries.lean`, and `AffineDifferenceOrbit.lean` [1, 2, 3]. This is a purposive sample of different proof mechanisms, not a random sample or an audit of the entire repository. In particular, no conclusion about every theorem imported by the repository is intended.

For Leant, the review covers relevant portions of the README and synthesis-internals document, and the verification scheduler implementation [4, 5, 6]. Repository test reports are useful context but are not reproduced experiments here. All Loom examples below are *proposed syntax*, not programs accepted by an existing compiler. Mathematical derivations are supplied explicitly; performance gains and end-to-end language soundness remain implementation and evaluation obligations.

## 1.2 What the sample actually teaches

The autonomous-equation module is already economical. Its main proof is recognizably an induction, and its helper theorem isolates a reusable argument. The residual friction is not bad mathematical organization: it is that the author must explicitly turn equality on an open set into equality near a point before differentiating [1].

The Abel-polynomial module carefully separates the polynomial definition, formal-series construction, coefficient extraction, and binomial identity. Its coefficient proof nevertheless interleaves the Lagrange-inversion argument with substitutions, casts, rational inverses, and factorial rearrangements [2]. Much of the latter work belongs in a reusable mathematical interface, not necessarily in a new search algorithm.

The affine-difference module proves a Boolean-cube expansion by induction. A large part of the successor case establishes the disjointness and injectivity needed to partition and reindex finite sums [3]. Those obligations are essential, but an author can describe their common structure more directly than a succession of low-level set rewrites.

These are three distinct opportunities: better *local reasoning*, better *representation management*, and better *certified transformations*. A language should support all three without conflating them with general theorem discovery.

## 2 Precedents and the proposed contribution

Readable formal proof languages have a substantial history. Isar treats a proof as a structured, executable document with explicit local contexts, rather than merely a sequence of interactions with a prover [11]. Mizar deliberately uses a mathematical vernacular that is both readable and formally checkable [12]. Naproche combines controlled mathematical language with logical representations and automated proof checking; its proof representation structures explicitly model discourse referents and nested assumptions [13, 14]. Thus, structured prose, mathematical notation, and scope-aware intermediate representations are precedents, not claimed inventions of this article.

Automation also supplies important precedents. Aesop organizes Lean proof search around rules and a white-box search process [15]. The Sledgehammer tradition separates external proof discovery from reconstruction inside the host prover [16]. Lean’s existing elaboration and tactic machinery already embodies a boundary between convenient surface instructions and proof terms [8, 9].

Leant adds a particularly relevant perspective. Its synthesis interface asks for an inhabitant of a type, while its interactive proof mode can suggest complete tactic combinations. It also documents the distinction between failed bounded search and genuine negative evidence in an adequately supported fragment [4]. This suggests that many small gaps in a mathematical proof should be presented as scoped synthesis problems rather than as demands that the author remember an exact lemma application.

LOOM’s proposed contribution is the combination of four design commitments. First, the document records *mathematical steps with contracts*, including domain and dependency information. Second, both term synthesis and domain-specific automation operate behind those contracts. Third, a *frozen formal statement and selected witnesses* prevent proof search from silently choosing a different interpretation. Fourth, a *replayable evidence layer* lets the reader expand each step, inspect its dependencies, and verify the finished result without rerunning discovery.

None of these ingredients alone is unprecedented. The research question is whether their integration gives a meaningfully better authoring interface for substantial Lean mathematics. That question requires comparison with strong existing practice, including the possibility that a library of helper lemmas and better editor support provides most of the benefit without a large new language.

### 3 A language of contracted mathematical steps

#### 3.1 A small core, extensible mathematical idioms

The core should be deliberately modest. It needs declarations, typed expressions, scoped assumptions, intermediate claims, witnesses, reductions, cases, induction, and calculation chains. Mathematical idioms such as differentiation and reindexing are extensions implemented by proof-producing *recipes*. They do not enlarge the logic.

```
theorem compose_at_point
  given A B C : Prop
  given f : A -> B -> C, g : A -> B, a : A
  show C
proof
  have b : B by g a
  exact f a b
end
```

Listing 1: Proposed core surface; expressions use Lean-style typed notation.

This example is intentionally unsurprising. Its elaborated term is

$$\lambda(A\ B\ C : \text{Prop})\ (f : A \rightarrow B \rightarrow C)\ (g : A \rightarrow B)\ (a : A). f\ a\ (g\ a).$$

The applications are justified by the declared types; the local proof  $b$  can be replaced by  $g\ a$ . The same result could be synthesized directly. The point is that writing an explicit intermediate claim and delegating it to synthesis should have the same scoped meaning.

The language should initially reuse Lean terms, binders, universes, notation, and declarations. A separate free-form mathematical parser would make semantic ambiguity a much larger problem. Presentation-oriented notation can be added through checked notation declarations, while unparsed explanatory prose remains a comment rather than an unacknowledged source of assumptions.

#### 3.2 The contract of a step

A step has more structure than its displayed sentence. It has a formal target, an ordered context of available objects and hypotheses, a permitted collection of definitions and inference rules, and any author-specified construction or transformation. Reconstruction may create additional obligations, but may not strengthen the theorem’s assumptions, change a chosen witness, or silently replace the target with an easier statement.

For example, “differentiate the identity on  $s$ ” has a contract involving the identity’s domain, the kind of derivative, and the point at which it is used. “Reindex by  $e$ ” has a contract involving the source and target index sets, the action of  $e$ , and the required multiplicity behavior. A generic `have B by search` has less mathematical structure, but still has the exact proposition  $B$  and a fixed context.

A recipe is an elaborator for this contract. It may find a theorem, instantiate parameters, generate subgoals, and propose proof terms. Its success is not authoritative: its output must be checked. The recipe can be buggy without being allowed to certify a false theorem. It can also be incomplete, in which case the author sees the remaining obligation rather than an invented justification.

### 3.3 Three views of one proof

The default *argument view* shows the human-authored claims and transformations. An *obligation view* shows the generated conditions, their scopes, and their status. An *evidence view* shows the exact proof terms or named auxiliary declarations that justify the conditions. These views are linked by source locations and stable step identifiers.

The system must not pretend that an automatically generated explanation is the original reason for a proof. A useful explanation is tied to the actual checked derivation. When a step says “by the chain rule,” the evidence view should identify the relevant chain-rule application and its hypotheses, not merely show that an unrelated global theorem happened to close the goal. This is an additional provenance policy, not a property obtained automatically from kernel checking.

### 3.4 Statement approval precedes proof search

Before searching for a proof, the system should display and freeze the elaborated theorem statement, including implicit binders, numerical domains, selected instances, and definitions referenced by notation. The user can keep a familiar mathematical rendering, but a fully explicit rendering must be available.

This rule prevents a particularly dangerous form of apparent intelligence: resolving ambiguous notation by choosing whichever meaning makes the theorem easiest to prove. Search is allowed to select between proofs of an approved proposition. It is not allowed to select the proposition itself without making the choice visible.

A statement hash is useful for detecting accidental changes, but it does not prove that the statement means what the author intended. Definitions, coercions, and imported assumptions are part of that meaning. The approval object must therefore include their identities and relevant environment information, not just the pretty-printed string.

## 4 Case study I: differentiating an induction identity

### 4.1 The original theorem and its essential hypotheses

The main declaration in `AutonomousIteratedDeriv.lean` is `AutonomousODE.iteratedDeriv_eq_comp [1]`. In mathematical notation its assumptions are as follows. Let  $s \subseteq \mathbb{R}$  be open, let  $W, \varphi : \mathbb{R} \rightarrow \mathbb{R}$ , and let  $G_n, H_n : \mathbb{R} \rightarrow \mathbb{R}$  be families indexed by  $n \in \mathbb{N}$ . Assume, for every  $x \in s$ ,

$$\text{HasDerivAt}(W, \varphi(W(x)), x), \quad (1)$$

$$G_0(W(x)) = W(x), \quad (2)$$

$$\text{HasDerivAt}(G_n, H_n(W(x)), W(x)), \quad (3)$$

$$G_{n+1}(W(x)) = H_n(W(x))\varphi(W(x)). \quad (4)$$

Here  $H$  is the family named  $G'$  in the source; using a separate letter avoids treating a supplied derivative witness as a globally defined derivative identity. The conclusion is

$$\forall n \in \mathbb{N} \forall x \in s, \quad D^n W(x) = G_n(W(x)). \quad (5)$$

The restriction in (3) matters. The theorem does not require  $G_n$  to be differentiable everywhere. It requires differentiability only at values  $W(x)$  reached from  $s$ . Likewise, no separate global smoothness assumption on  $W$  is inserted. A purported simplification that added these stronger hypotheses would change the theorem, not merely shorten its proof.

The original Lean proof introduces the induction and then explicitly constructs an eventual equality using `Filter.eventuallyEq_of_mem`, `hs.mem_nhds hx`, and the induction hypothesis. It rewrites derivatives using `heq.deriv_eq`, constructs a chain-rule witness with `HasDerivAt.comp`, and substitutes the recurrence. These are precise implementations of a familiar local argument [1].

## 4.2 A proposed argument-level proof

Let the assumptions above have the names `hW`, `h0`, `hG`, and `hstep`, with `hs` naming openness. A proposed proof body is:

```
induct n generalizing x
base
  fix x in s
  show D[0] W x = G[0](W x) by definition, h0
step n ih
  fix x in s
  locally at x on s using hs
  differentiate ih using hW, hG
  show D[n+1] W x = G[n+1](W x) by hstep
```

Listing 2: Proposed Loom proof of the frozen statement (5).

The notation  $D[n]$  is a declared abbreviation for Lean’s iterated derivative, not an informal operator with unspecified semantics. The induction motive, visible on demand, is

$$P(n) := \forall x : \mathbb{R}, x \in s \rightarrow D^n W(x) = G_n(W(x)).$$

The variable  $x$  remains arbitrary in the induction hypothesis. The phrase `locally at x on s` is an instruction to build the neighborhood evidence; it is not a new hypothesis.

## 4.3 The complete mathematical reconstruction

For  $n = 0$ , the definition of the zeroth iterated derivative gives  $D^0 W(x) = W(x)$ , and (2) gives the conclusion. Suppose the identity holds for  $n$  at every point of  $s$ . Fix  $x \in s$ . Since  $s$  is open, it is a neighborhood of  $x$ . The induction hypothesis therefore gives

$$D^n W = G_n \circ W \quad \text{on a neighborhood of } x. \quad (6)$$

The chain rule, using precisely (1) and (3), gives

$$D(G_n \circ W)(x) = H_n(W(x))\varphi(W(x)). \quad (7)$$

Local equality permits transfer of the derivative in (7) to  $D^n W$ . Hence

$$D^{n+1} W(x) = H_n(W(x))\varphi(W(x)) = G_{n+1}(W(x)),$$

where the last equality is (4). This proves the induction step.

The recipe’s obligation ledger is consequently small but substantive. It must establish  $s \in \mathcal{N}(x)$ , instantiate the induction identity at arbitrary nearby points, construct the two derivative witnesses, compose them in the correct order, and transport the resulting derivative through the local equality. It need not first prove

both sides differentiable independently: a derivative witness for one side can be transferred across eventual equality. Finally it must identify the successor iterated derivative and check the recurrence's orientation.

This reconstruction is not made valid by the word “differentiate.” It is valid because the word expands to a certified instance of this argument. A recipe can expose the same result as a reusable theorem, and the language should prefer such a theorem when it matches. Syntax does not replace mathematical API design.

#### 4.4 What a correct implementation must reject

Equality at a point is insufficient. The functions  $f(t) = 0$  and  $g(t) = t$  agree at 0, while  $f'(0) = 0$  and  $g'(0) = 1$ . Thus a step that offers only  $f(x) = g(x)$  cannot justify differentiating the equality at  $x$ .

Equality on an arbitrary set is insufficient for an ordinary derivative for the same reason: take the set  $\{0\}$ . The diagnostic should identify the missing neighborhood relation, not merely report that a tactic failed. A useful message would say that the supplied equality holds on  $s$ , but neither openness at the chosen point nor another neighborhood witness has been established.

Derivatives within a set require a different recipe with a different contract. In particular, ordinary derivatives, one-sided derivatives, and `HasDerivWithinAt` statements must not be interchanged because their printed formulas resemble one another. The appropriate domain and uniqueness conditions belong to the recipe. This is where a mathematically literate interface must be more precise than casually written prose, while still sparing the author the routine encoding of that precision.

### 5 Case study II: Lagrange inversion without cast choreography

#### 5.1 A statement over a rational algebra, not a field

The source declaration `Fabius.coeff_exp_subst_of_abel_eq` works over a commutative ring  $A$  equipped with a  $\mathbb{Q}$ -algebra structure [2]. Let

$$[q]_A := \text{algebraMap}_{\mathbb{Q}, A}(q), \quad q \in \mathbb{Q},$$

and write  $m_A$  for the natural-number scalar in  $A$ . Define the formal exponential

$$E_a(t) = \sum_{k \geq 0} [(k!)^{-1}]_A a^k t^k \in A[[t]].$$

Suppose the formal series  $T$  satisfies

$$T(t) = tE_{-a}(T(t)). \tag{8}$$

For every  $m \geq 1$  the desired identity is

$$[t^m]E_x(T(t)) = [(m!)^{-1}]_A x(x - m_A a)^{m-1}. \tag{9}$$

The source uses  $m = n + 1$ , so positive degree is built into the statement. It proves the result for any solution of (8), not only the canonical solution constructed elsewhere in the module.

The notation above prevents a semantic mistake. A fraction in (9) is a rational scalar transported into  $A$ ; it is not arbitrary division in  $A$ . Nothing here assumes that  $A$  is a field, an integral domain, or even nontrivial. A normalization step that silently changed the ambient category to fields would be unsound as a translation of this theorem.

## 5.2 The mathematical proof

First, (8) implies  $[t^0]T = 0$ , which supplies sufficient admissibility evidence for the substitutions used here. The formal exponentials satisfy

$$E_{-a}(u)E_a(u) = 1, \quad E_b(u)E_c(u) = E_{b+c}(u), \quad E'_x(u) = xE_x(u).$$

These are formal-series identities; no analytic convergence assertion is being made. The Lagrange–Bürmann coefficient identity, in a denominator-free form before scalar cancellation, yields

$$\begin{aligned} m_A[t^m]E_x(T(t)) &= [u^{m-1}]E'_x(u)E_{-a}(u)^m \\ &= [u^{m-1}]xE_{x-m_Aa}(u) \\ &= x[((m-1)!)^{-1}]_A(x - m_Aa)^{m-1}. \end{aligned} \tag{10}$$

These are the mathematical ingredients applied in the inspected proof through the local Lagrange-inversion and exponential-series API [2].

Since  $m$  is a positive integer,  $m^{-1}m = 1$  in  $\mathbb{Q}$ . Applying the algebra map gives

$$[m^{-1}]_A m_A = 1_A. \tag{11}$$

Multiplying (10) by  $[m^{-1}]_A$  and using

$$[m^{-1}]_A[((m-1)!)^{-1}]_A = [(m!)^{-1}]_A$$

proves (9). The use of a rational inverse is justified before transport. Injectivity of the algebra map is unnecessary; preservation of multiplication and of 1 is enough.

## 5.3 The argument-level document

With  $m = n + 1$  and the formal-series notation fixed, the proposed proof can read:

```
have subst_ok : admissible_substitution T
  from hT by the zero_constant_term criterion

have coeff_identity
  by Lagrange_inversion on hT
  with outer_series := E(x), degree := n+1

normalize coeff_identity
  using formal_exponential_rules, rational_algebra(A), factorials

show coefficient(n+1, E(x).subst(T)) =
  scalar(A, 1/(n+1)!) * x * (x - nat_scalar(A,n+1)*a)^n
  from coeff_identity
```

Listing 3: Proposed Loom proof; rational fractions denote transported scalars.

Names such as `Lagrange_inversion` and `rational_algebra(A)` denote versioned recipes with declared theorem dependencies, not English phrases sent to an unrestricted oracle. The recipe must expose which Lagrange-inversion theorem was instantiated, how its hypotheses were supplied, and how the final scalar identity was checked.

The original source constructs  $hs$ ,  $hinverse$ , a coefficient identity  $h$ , and the auxiliary equalities  $hcancel$  and  $hfactorial$ , then assembles a calculation [2]. The proposed surface preserves their logical roles but moves routine library representation changes into recipes. It does not claim that these obligations disappear or that current Lean cannot already automate some of them.

## 5.4 Normalization must be typed and local

The normalization recipe should have a deliberately narrow scope. It can move a natural scalar through an algebra map, combine rational scalar factors, normalize polynomial expressions over a commutative ring, and unfold the factorial recurrence at a positive index. It cannot invoke general cancellation of a possibly nonunit element of  $A$ .

A good intermediate representation distinguishes a natural index, its cast into  $\mathbb{Q}$ , its cast into  $A$ , and a formal-series constant with that coefficient. Their printed notation can be light, but their types and conversion paths must remain explicit internally. Equality certificates accompany every nondefinitional conversion.

This also guards against a frequent source of accidental changes: natural subtraction. The exponent  $m - 1$  in (9) is a natural number with  $m \geq 1$  established. The term  $x - m_A a$  is subtraction in  $A$ . The same minus sign in the document cannot license rewriting both with the laws of a field.

A missing positivity hypothesis should produce an obligation such as  $m \neq 0$  in  $\mathbb{Q}$  rather than an unexplained timeout. If the theorem is extended to  $m = 0$ , the compiler must create a separate case; the constant coefficient is 1, and the positive-degree formula is not the right presentation of that case.

## 5.5 A second level of compression: a generating-function argument

The same module proves the Abel binomial identity by multiplying exponential generating functions and extracting coefficients [2]. With

$$A_0(a, x) = 1, \quad A_m(a, x) = x(x - m_A a)^{m-1} \quad (m \geq 1),$$

its established series identity gives

$$\sum_{m \geq 0} [(m!)^{-1}]_A A_m(a, x) t^m = E_x(T(t)).$$

Multiplying the identities for  $x$  and  $y$  and using  $E_x(T)E_y(T) = E_{x+y}(T)$  yields

$$A_n(a, x + y) = \sum_{k=0}^n \binom{n}{k}_A A_k(a, x) A_{n-k}(a, y),$$

where  $\binom{n}{k}_A$  denotes the natural binomial coefficient mapped into  $A$ .

Here the right abstraction is not “find shorter rewrites.” It is an EGF interface that knows coefficient extraction, multiplication, and the factorial normalization convention. A proof command such as “compare EGF coefficients” should call that interface. Library-level conceptual compression and language-level compression should reinforce one another rather than compete.

## 6 Case study III: a finite sum over the Boolean cube

### 6.1 The algebraic theorem

In `AffineDifferenceOrbit.lean`, the declaration `Fabius.affineDifference_iterate_apply` expands iterates of

$$(\Delta_{b,c}f)(z) = f(bz + c) - f(bz - c).$$

Its algebraic assumptions are only that  $K$  is a commutative ring and  $E$  is an additive commutative group, with  $f : K \rightarrow E$  [3]. Define

$$I_n = \{0, \dots, n-1\}, \quad p_n(T, z) = b^n z + c \sum_{k \in I_n} b^k - 2c \sum_{k \in T} b^k.$$

The theorem is

$$(\Delta_{b,c}^n f)(z) = \sum_{T \subseteq I_n} (-1)^{|T|} \bullet f(p_n(T, z)), \quad (12)$$

where the scalar action is the canonical integer action on  $E$ . This finite theorem uses neither differentiation nor division. The source subsequently builds analytic results on top of it, but those stronger assumptions should not migrate backwards into (12).

### 6.2 The proof a mathematician wants to maintain

The key decomposition is

$$\mathcal{P}(I_{n+1}) \cong \mathcal{P}(I_n) \sqcup \mathcal{P}(I_n). \quad (13)$$

The first component consists of sets not containing  $n$ ; the second consists of sets containing  $n$ . Their parametrizations are  $U \mapsto U$  and  $U \mapsto U \cup \{n\}$  respectively. The inverse on the second component removes  $n$ .

For  $U \subseteq I_n$ , the definitions give

$$p_{n+1}(U, z) = p_n(U, bz + c), \quad (14)$$

$$p_{n+1}(U \cup \{n\}, z) = p_n(U, bz - c), \quad (15)$$

$$(-1)^{|U \cup \{n\}|} = -(-1)^{|U|}. \quad (16)$$

The first two identities follow by expanding  $\sum_{k \in I_{n+1}} b^k = \sum_{k \in I_n} b^k + b^n$ ; the second also uses  $\sum_{k \in U \cup \{n\}} b^k = b^n + \sum_{k \in U} b^k$ . The third follows from  $n \notin U$ .

The base case of (12) has the single set  $\emptyset$  and is immediate. In the successor case, expand the outer affine difference and use the induction hypothesis at  $bz + c$  and  $bz - c$ . Decomposing the desired right-hand side by (13), then applying (14)–(16), gives exactly that difference of sums. This proves the theorem.

```

induct n generalizing z
base
  fix z
  normalize using empty_powerset, affine_point
step n ih
  fix z
  expand the outer affine_difference
  use ih at  $b*z+c$  and  $b*z-c$ 
  split target_sum by (n in T)
  case absent
    reindex T by  $U := T$ 
  case present
    reindex T by  $U := \text{erase}(T,n)$ 
    inverse U := insert(n,U)
  normalize using affine_point_step, cardinality_insert,
    integer_action, finite_sum_rules

```

Listing 4: Proposed Loom proof exposing the partition and the changed arguments.

The proof still records the important choices: induction, generalization over  $z$ , the membership partition, and the inverse map. The generated work includes the finite-set side conditions and additive-group rearrangements. An implementation may package the whole partition as a certified equivalence rather than perform separate tactic rewrites, which is likely to make the reconstruction more stable.

### 6.3 The obligations hidden by “reindex”

A finite sum is not a bag of syntactic summands that may be arbitrarily renamed. For a bijective reindexing, the recipe must check that the proposed forward map lands in the target domain, that its inverse lands in the source domain, and that both composites are identities on their stated domains. It must then establish equality of the corresponding summands.

In (13), coverage follows by deciding whether  $n \in T$ . Disjointness follows because the two branches assert opposite membership facts. If  $U \subseteq I_n$ , then  $n \notin U$ ; hence removing  $n$  after inserting it recovers  $U$ . Conversely, if  $n \in T \subseteq I_{n+1}$ , then reinserting  $n$  after removing it recovers  $T$ , and the remainder is a subset of  $I_n$ . These facts also justify the cardinality change.

The source proves the required disjointness and uses equality after erasing  $n$  to establish the injectivity needed by `sum_image` [3]. The new language should encapsulate this argument, not pretend it is unnecessary. A collision-producing map is a useful negative test: mapping both elements of  $\{0, 1\}$  to 0 does not preserve a sum of two ones as a sum over its one-element image. A nonbijective transformation needs a different theorem that explicitly accounts for fibers or multiplicities.

### 6.4 Why the induction generalization cannot be implicit guesswork

The induction hypothesis is needed at two new arguments,  $bz + c$  and  $bz - c$ . A hypothesis only about one fixed  $z$  will generally not suffice. Consequently, generalizing  $z$  is not incidental syntax: it records a substantive property of the induction motive.

The system can suggest this generalization after analyzing the failed successor step, but accepting that suggestion should update the displayed motive. It must not silently generalize dependent assumptions in a

way that changes their meaning or loses a necessary relation. Motive synthesis is a useful assistant, not an invisible authority.

## 6.5 A shared recipe across the two analytic examples

The affine-difference module also proves, on an open domain preserved by both affine branches, that a one-step identity

$$f'(z) = a \bullet (\Delta_{b,c} f)(z)$$

propagates to

$$D^n f(z) = (a^n b^{\binom{n}{2}}) \bullet (\Delta_{b,c}^n f)(z). \quad (17)$$

The source uses neighborhood equality, the chain rule, scalar compatibility, and  $\binom{n+1}{2} = \binom{n}{2} + n$  [3]. Thus the locality recipe from Section 4 applies again, with separate obligations that the affine branches remain in the domain. This is a concrete reason to make recipes reusable mathematical interfaces rather than one-off compression macros.

The dyadic specialization in the same file supplies another useful guardrail: the affine argument  $2^n z + 2^n - 1 - 2j$  is evaluated in the coefficient ring, not by truncated subtraction in  $\mathbb{N}$  [3]. A readable notation layer must retain that distinction even when suppressing explicit casts.

## 7 What the language should infer, and what it should expose

### 7.1 Mathematical working contexts

Mathematicians rarely reason in a flat list of unrelated facts. They work on an open neighborhood, in a coefficient ring, under an induction hypothesis, or inside one branch of a decomposition. Loom should represent these *working contexts* explicitly. A context is not a heuristic license to assume anything customary in the subject. It is a scoped collection of typed objects, proven refinements, and declared inference recipes.

For instance, a finite-sum context may register  $U \subseteq I_n$  and derive  $n \notin U$  when requested. A rational-algebra context records the algebra map and its laws. A local-calculus context records the actual filter and the evidence that a domain belongs to it. The context manager can index these facts by mathematical role so that a recipe does not repeatedly rediscover them from a large unstructured environment.

There should be no automatic strengthening from “continuous” to “smooth,” from “commutative ring” to “field,” or from “equality almost everywhere” to “pointwise equality.” Facts derived inside a case remain inside that case unless exported through a valid case-elimination proof. The language’s apparent naturalness should come from well-designed contexts, not permissive interpretation.

### 7.2 Inference should follow a hierarchy of risk

At the lowest risk level are uniquely determined syntactic arguments and definitional reductions. Next come certified representation changes, such as transporting a scalar through a specified homomorphism. Then come routine logical connections and specialized calculations. At the highest level are mathematical choices: a witness, an induction motive, a case partition, or an auxiliary invariant.

This is not a semantic hierarchy of truth: every accepted result requires evidence. It is an interface hierarchy. The first levels can usually proceed silently with expandable provenance. The last level should normally be shown as a choice, even when an engine can propose it. A witness for a pure existential proposition may be

freely constructed if the entire predicate is proved; a witness defining later computational behavior is often important to the author and should be recorded visibly.

A universal command obvious would blur these distinctions. A better facility is by `routine`, interpreted by a bounded, locally declared policy. When it fails, the system should report whether it lacks a theorem, a side condition, a representation bridge, or a mathematical choice. The author can then supply a missing claim rather than debug a hidden search strategy.

### 7.3 Controlled omission, not semantic guessing

An omitted argument may be inferred when its constraints select a unique approved interpretation, or when the omission is expressly a request to synthesize a proof of a fixed proposition. An ambiguous computational term is different. From a type  $A \rightarrow A \rightarrow A$ , both projections are available; from a state-transformer type, multiple state-threading behaviors may be available. Leant’s examples explicitly illustrate that type-correctness alone need not specify the intended computation [4].

Pronouns such as “this identity” can be convenient, but resolution must be typed and local. A unique salient equality may be selected. If two distinct identities fit, the system should show the alternatives rather than pick the one that yields the easiest proof. The chosen referent becomes a stable identifier in the elaborated document, so later insertion of a sentence cannot silently redirect it.

Likewise, “similarly” should mean either an explicitly indicated parameter substitution in a checked proof schema or a request for a suggested analogous derivation. It is not an inference rule. “By symmetry” needs a transformation preserving the hypotheses and transporting the conclusion. “Without loss of generality” additionally needs coverage or an appropriate reduction theorem. These idioms belong in later recipes with visible contracts, not in an unqualified first version of the core.

### 7.4 Witnesses and quantifier scope

A particularly important distinction is between

$$\forall x : X \exists y : Y, R(x, y) \quad \text{and} \quad \exists y : Y \forall x : X, R(x, y).$$

In a proof of the first, a local witness may depend on  $x$ . It cannot escape its scope as the uniform witness required by the second. The language must preserve the dependency of a chosen term, even if both proofs are narrated as “choose  $y$ .”

The rule should also distinguish a proof that something exists from the construction of data. Lean’s restrictions on eliminating propositions into data and its treatment of choice are part of the host logic, not details that a front end may bypass [7]. A construction can provide a dependent pair directly. Extracting a choice function from purely propositional existence may require an explicit classical policy. The editor should show which happened.

An illustrative declaration is:

```
choose y := candidate(x) satisfying R(x,y)
by witness_lemma using hx
```

Listing 5: Proposed witness syntax: a term and its specification remain linked.

Here the author chose the construction, and the proof obligation concerns that exact term. In the alternative `choose y satisfying R(x,y) by search`, the engine may propose both a witness and a proof, but the selected pair must be retained. Replacing the witness later by a smaller term of the same type invalidates the old specification proof unless a new proof or an appropriate transport is supplied.

## 8 A scoped obligation calculus

### 8.1 Objects and judgments

A rigorous design needs a distinction between a proposed surface syntax and its accepted meaning. Let  $\Sigma$  be a well-formed environment of typed declarations. Let

$$\Gamma = x_1 : A_1, \dots, x_k : A_k(x_1, \dots, x_{k-1})$$

be an ordered dependent telescope. A local goal is a judgment  $\Sigma; \Gamma \vdash A : \text{Sort } u$ . Some goals ask for proofs in Prop; others ask for terms used in constructions. The implementation must not treat all holes as proof-irrelevant.

Let  $\Pi$  be an elaboration and reconstruction policy: visible declarations, permitted recipes, resource limits, and an axiom policy for final acceptance. A surface step  $d$  elaborates schematically as

$$\Sigma; \Gamma; \Pi \vdash d \rightsquigarrow (e : A, O, M), \quad (18)$$

where  $e$  is an expression with typed holes,  $O$  is a collection of obligations, and  $M$  records source locations and author commitments. This judgment describes a proposed elaborator; it is not a new kernel typing rule.

Each obligation has an identifier, its own ordered telescope, a target type, dependency edges, an origin step, a status, and optional search guidance. Its telescope includes the free variables actually needed to interpret the goal and any parameters on which they depend. Keeping only a pretty-printed list of propositions is inadequate: it would lose binder identity, universe information, and dependencies between witnesses.

The implementation should distinguish expression holes, type and universe constraints, instance-resolution problems, and ordinary proof obligations. A proof cannot be declared complete while an unresolved type choice remains hidden in a side channel.

### 8.2 Ordinary proof rules remain ordinary

Most core constructs elaborate to elementary type-theoretic operations. Suppose

$$\Sigma; \Gamma \vdash p : B, \quad \Sigma; \Gamma, h : B \vdash t : A.$$

Then “have  $h : B$  by  $p$ ; continue with  $t$ ” produces

$$(\lambda h : B. t) p : A.$$

When  $A$  depends on  $h$ , the resulting type is the corresponding substitution; the compiler may not discard that dependency. A nondependent mathematical proof usually hides this complication, but the intermediate representation must support it.

A reduction “it suffices to prove  $B$ ” requires a bridge  $r : B \rightarrow A$ . After obtaining  $b : B$ , the result is  $r b$ . Proving  $A \rightarrow B$  instead does not justify the reduction. This simple orientation check is one reason to represent the reduction explicitly rather than as a prose annotation.

Choosing a witness  $w : X$  for  $\exists x : X, R(x)$  creates the exact obligation  $R(w)$ ; a proof  $p : R(w)$  gives  $\text{Exists.intro } w \ p$ . Universal introduction extends the telescope by a fresh variable. Case analysis uses the appropriate eliminator, with branch-local variables and hypotheses. A calculation chain uses certified transitivity or compatibility theorems for its actual relations; equality, inequality, equivalence, and asymptotic comparison are not interchangeable punctuation.

Induction creates a motive and elaborates through an existing induction principle. The motive must account for all generalized variables and dependent hypotheses. There is no unrestricted rule allowing an obligation to cite itself. Recursion, mutual induction, or well-founded reasoning is accepted only through the host’s checked principles and their required termination or well-foundedness evidence.

### 8.3 A proof graph is not merely a dependency diagram

A visible argument may be a directed acyclic graph of intermediate claims, but the implementation also has a constraint graph. Two apparently separate goals may share a metavariable. Solving one can determine a witness, type, or universe used by the other. They are not independent tasks merely because they occupy different boxes in an editor.

A scheduler must either preserve the shared constraints transactionally or close each task over a well-defined interface of parameters. A worker cannot resolve a shared witness one way while another worker assumes a different resolution. A solution should be committed together with the substitution it induces, after checking consistency with the current state.

Likewise, a cached proof of  $B$  under  $x : X$  cannot be copied into a context containing an unrelated variable also printed as  $x$ . Reuse requires a checked context substitution. In the dependent case this substitution must map each variable to a term of the appropriately substituted type, in telescope order.

**Lemma 8.1** (Scoped substitution). *Suppose  $\Sigma; \Delta \vdash p : B$  and  $\theta$  is a well-typed substitution from the ordered telescope  $\Delta$  into  $\Gamma$ . Then*

$$\Sigma; \Gamma \vdash p[\theta] : B[\theta].$$

*Proof.* Proceed by induction on the typing derivation for  $p$ . A variable is replaced by the term supplied by the well-typed substitution. Applications follow from the induction hypotheses for the function and its argument. Under a binder, rename the bound variable freshly and extend the substitution by that variable; the induction hypothesis applies to the body. Constants retain their declared types, and the remaining constructors use the corresponding substitution instances of their typing rules. In particular, the type substituted for each dependent variable must be computed after substituting its predecessors. This is the usual substitution argument, stated here because it is exactly the obligation a proof cache or external worker must respect.  $\square$

This lemma justifies reuse under a valid context map. It does not justify identifying contexts by text, or replacing one witness by another with the same ambient type.

### 8.4 Checking a reconstruction before all holes are solved

A useful implementation technique is to check a *conditional proof skeleton*. Instead of filling holes with sorry, abstract them as explicit parameters. If a calculation requires  $h_1 : B_1$  and  $h_2 : B_2(h_1)$ , check a term of the form

$$K : \prod_{h_1 : B_1} \prod_{h_2 : B_2(h_1)} A(h_1, h_2).$$

The skeleton then records a valid conditional derivation, not a completed theorem. Actual solutions can be substituted in dependency order. An unresolved parameter is visibly an outstanding obligation, and a final result is accepted only when the original theorem is obtained without adding new assumptions to it.

Not every raw elaboration state immediately admits this form: unresolved type constraints may first need solving, and ill-scoped cyclic dependencies must be rejected. Where it applies, however, abstraction makes the gap between a valid plan and a completed proof precise. It also enables recipe testing without pretending that failed leaves are proven.

## 8.5 Conditional soundness of the proposed architecture

**Proposition 8.2** (Contract-preserving assembly). *Fix an approved target  $A$  in  $\Gamma$  and an environment  $\Sigma$ . Suppose:*

1. *a reconstruction skeleton is checked under explicit, well-scoped obligation parameters;*
2. *each parameter is filled, in dependency order, by a checked term of its exact substituted type;*
3. *the assembled result is checked against the approved target, and its transitive axiom dependencies satisfy the chosen policy.*

*Then the architecture accepts a derivation of  $A$  in  $\Gamma$  relative to  $\Sigma$  and that policy, without treating search success or a natural-language phrase as an additional inference rule.*

*Proof.* Each replacement of an obligation parameter is justified by scoped substitution. Repeating the replacements produces a term of the skeleton’s instantiated result type. The final check establishes that this type is the approved  $A$ , with all external variables accounted for by  $\Gamma$ . The axiom audit establishes the separate dependency condition. Search procedures and recipes affect which terms are proposed, but no step in this reasoning uses their assertions as proof evidence.  $\square$

This is a conditional architectural argument, not a machine-checked theorem about an implemented LOOM compiler. Indeed, a final independent check can reject a malformed skeleton or a faulty assembler. The important design consequence is that a synthesizer need not be trusted for logical validity, while the correspondence between the approved target and the author’s intended mathematics still requires separate attention.

## 8.6 Four distinct correctness questions

The architecture should expose four questions rather than hide them behind one green check. *Typing correctness* asks whether the emitted term proves the emitted proposition. *statement fidelity* asks whether that proposition is the approved one under the intended definitions. *contract fidelity* asks whether a step used the designated witness, domain, or reduction. *explanatory fidelity* asks whether its displayed reason accurately describes the checked argument.

The first can be handled by proof checking. The second needs a stable target and semantic review. Much of the third can be encoded as exact types, dependency constraints, and term identities. The fourth is only partly formal: a system can verify which theorem was applied, but that does not measure whether the explanation is enlightening. A language aimed at mathematicians should improve all four without confusing them.

# 9 Making Leant a reconstruction service

## 9.1 What should be reused

Leant already provides an experimental synthesis REPL around Lean and Djex. Its documented workflow translates a supported type problem, searches for inhabitants, renders candidates, and checks candidates against the original Lean goal. Its proof mode can suggest tactic sequences, including combinations involving induction, simplification, and arithmetic [4]. These are useful services for a new proof document; they are not yet the whole document model proposed here.

Three aspects are especially relevant. Structural term synthesis can reconstruct applications, projections, currying, and other logical connections between available facts. Ranked alternatives can expose genuinely

different constructions instead of committing to the first inhabitant. The distinction between a supported complete fragment and bounded heuristic search can prevent misleading negative conclusions [4, 5].

The integration should respect those boundaries. Treating an arbitrary dependent analysis goal as though it were fully represented by a structural type problem would overstate the engine’s coverage. A lossier representation may still be useful for proposing a candidate, provided the candidate is checked against the exact original goal. It does not automatically support a negative verdict about that goal.

## 9.2 A proposed request and response boundary

A request should contain the exact Lean target, the local telescope, the selected environment snapshot, a finite inventory of relevant providers, the recipe or generic search policy, and the resource budget. An implementation hosted inside Lean can retain expressions directly. An external adapter can close the telescope into a dependent function type and then instantiate a returned term in the original context. Any text round trip must be re-elaborated under the frozen target, not under newly inferred free variables.

The result is not just a string. It should retain the candidate expression or exact candidate text, the request identity, provider instantiations, its verification status, and the evidence appropriate to that status. Search alternatives may be ranked, but the evidence must move together with its candidate.

Leant’s internal documentation describes a semantic-origin record and provider bindings that retain source and search information, and it emphasizes preserving candidate/evidence associations through rendering and selection [5]. That is a strong conceptual precedent. Loom would extend the association upward to the originating mathematical step and its approved contract.

```
request := {
  target      : ExactLeanType,
  context     : DependentTelescope,
  environment : SnapshotId,
  providers   : ApprovedProviderInventory,
  commitments : WitnessesAndTransformations,
  budget      : SearchBudget
}

result := Candidate(request_id, expression, provenance)
  | Unsupported(reason)
  | Unknown(reason, explored_budget)
  | FragmentRefutation(fragment, certificate, adequacy)
```

Listing 6: Proposed adapter interface; not an existing Leant API.

A separate acceptance component then checks the candidate. The last alternative is deliberately demanding: a fragment refutation is meaningful for the original problem only when the translation and available-premise inventory support that inference. Otherwise the appropriate result is Unknown or a narrowly labeled statement about the fragment alone.

## 9.3 Why verification statuses must be distinct

The inspected `src/Leant/Synth/Verification.hs` defines an opaque `Verified` candidate receipt. Its documentation explicitly says that the receipt records acceptance by a supplied callback, not behavioral evidence, a solver certificate, or a kernel proof. The keyed scheduler records accepted spellings, permits

retries after rejection, and does not transfer semantic authority between candidate occurrences [6]. A front end must preserve that precision rather than interpreting the type’s name as a stronger guarantee.

The proposed interface distinguishes the following states:

| State                | What has actually been established                                                      |
|----------------------|-----------------------------------------------------------------------------------------|
| Proposed             | A search process returned a candidate.                                                  |
| Elaborated           | The candidate was interpreted against the specified Lean type.                          |
| Callback accepted    | A particular verifier callback reported acceptance.                                     |
| Replayed             | The exact declaration or term was independently checked under the recorded environment. |
| Specification proved | A proof of the stated predicate for the selected witness was checked.                   |
| Unknown              | The service did not establish a result within its supported fragment or budget.         |

These are not interchangeable labels, and some are independent dimensions rather than a universal linear progression. A proof of an existential theorem can already establish its full proposition. A type-checked program, by contrast, may need an additional specification proof. A replayed proof can still depend on axioms that the project does not permit.

## 9.4 A bounded reconstruction portfolio

A practical default should use a small deterministic sequence of increasingly expensive attempts. First resolve already determined arguments, explicit assumptions, and direct theorem applications. Next perform declared normalization and structural synthesis. Then run specialized procedures appropriate to the goal: arithmetic, polynomial identities, propositional search, or a domain recipe. Finally, larger theorem search or model-guided proposals may be attempted under an explicit budget.

The exact order is a design parameter to evaluate, not a universal prescription. A recipe that knows it is solving a coefficient identity should not spend its budget attempting irrelevant induction schemes. Conversely, an engine should not keep unfolding every definition merely because that might eventually expose a tactic’s preferred syntax.

Resource limits should include attempted expansions, heartbeats or comparable prover limits, wall-clock deadlines, memory, and candidate-verification quotas. A success quota alone does not bound a stream with infinitely many failures. Leant’s verification implementation explicitly leaves stream bounding to its caller [6]; the document-level scheduler needs its own complete resource policy.

When all attempts fail, the remaining goal is a first-class artifact. The author can introduce a lemma, select a witness, open a Lean escape block, or revise the intended strategy. The interface should distinguish “unsupported dependency,” “missing nonzero condition,” and “no proof found” rather than collapsing them into a mysterious red mark.

## 9.5 Ranking explanations, not just terms

Leant’s configurable candidate-quality profiles are evidence that ranking is itself a significant part of synthesis engineering [4, 5]. For proof documents, the ranking objective should also account for explanatory structure and maintenance costs.

One proposed objective for a completed reconstruction  $r$  is

$$J(r) = \alpha L(r) + \beta S(r) + \gamma U(r) + \delta F(r),$$

where  $L$  measures author-maintained detail,  $S$  measures reconstruction cost,  $U$  measures unsupported or unexplained jumps in the displayed argument, and  $F$  estimates dependency fragility. These quantities require operational definitions and empirical calibration. The formula is a design objective, not a claim that the system can measure mathematical insight.

A shortest-term ranking can prefer an opaque imported theorem over a transparent argument. A shortest-script ranking can hide arbitrarily expensive search. A shortest-document ranking can move all complexity into a new helper library. All three may be legitimate engineering choices, but none is an honest measure of reduced authoring burden by itself.

The user should therefore be offered a few qualitatively distinct alternatives when choices matter: a direct application, an elementary calculation, or an induction with a visible invariant. Routine alternatives that differ only in lambda-bound variable names need not clutter the interface.

## 9.6 Behavioral constraints and negative evidence

Leant documents a separate, restricted Length-assessment layer that can use replayed counterexamples to rank or filter certain already type-checked program candidates. It also distinguishes such evidence from unrestricted proof of a behavioral specification [4, 5]. The same discipline should govern Loom.

Testing an invented witness can expose a mistake or improve its ranking. It cannot by itself discharge an unbounded mathematical assertion. A solver’s answer becomes proof evidence only through a checked translation and an accepted certificate or reconstruction. Bounded success should be labeled with its bound; unsupported translations should fail explicitly; absence of a found counterexample should not be presented as a universal theorem.

For synthesis, positive and negative transfers are asymmetric. A candidate obtained from an approximation can be checked directly in the original problem. A failure in the approximation does not refute the original without a justified adequacy result. This principle is crucial when extending Leant’s structural capabilities to the richer dependent contexts of mathematical proofs.

# 10 Proof artifacts, reproducibility, and the trust boundary

## 10.1 Freeze discovery; replay evidence

There should be two modes of work. In *discovery mode*, recipes and search engines may explore alternatives, query theorem indices, or ask a model for a proposed step. In *replay mode*, the system loads the selected evidence and checks it without repeating that exploration. A finished proof document should normally ship in replayable form.

A frozen artifact should record the approved theorem type, its universe parameters and local context interface, the selected proof term or auxiliary declarations, selected computational witnesses, exact dependency identities, recipe versions, environment and toolchain information, source-to-evidence links, and the transitive axiom inventory. The artifact can include the readable argument and a suggested tactic script, but neither substitutes for the checked evidence.

A tactic script is a recipe for regenerating a proof; it is not necessarily a stable representation of the already found proof. A proof term is more precise, but its serialization may depend on toolchain versions. The proposal is therefore reproducible replay in an explicitly supported environment, not a promise that every future Lean version will consume today’s internal binary format unchanged.

## 10.2 Meaning, validity, and foundations are separate

Lean’s official validation guidance distinguishes proving the intended statement from validating a formal proof, and discusses axiom auditing and additional checking tools [10]. Loom should make that distinction central to its artifact format. A proof of a mistranslated statement is not the requested mathematical result. A valid conditional theorem may also be misrepresented if its assumptions are hidden in the presentation.

Final acceptance should require that the checked result has exactly the approved target, with no unresolved holes and no unexpected extra parameters. It should also audit transitive axiom dependencies against a project policy. The policy can explicitly permit ordinary classical foundations, or impose a more restrictive constructive discipline; it should not merely search for the literal word *sorry* in the source.

Native computation needs version-sensitive treatment. The current Lean validation documentation notes that recent releases can represent compiler-based computation through dedicated axioms, rather than relying only on the older global mechanism [10]. A strict replay profile should consequently allowlist actual axiom dependencies, not blacklist a single historical name. A broader trust profile is possible, but its additional assumptions must be visible.

## 10.3 Untrusted search still needs operational containment

Keeping an engine outside the logical trust boundary does not make arbitrary generated code operationally harmless. Lean metaprograms and build steps can perform effects; a malicious suggestion may attempt to read files or alter the environment rather than simply construct a term. The official validation guidance explicitly considers adversarial proof-production settings [10].

The proposed architecture therefore separates the discovery process from the final checker. Discovery runs in a restricted environment with controlled imports, file access, network access, and resource limits. The checker receives a narrowly specified artifact and a trusted target in a clean process. Its input parser and any imported binary format must themselves be treated as part of the operational attack surface.

An independent checker can further diversify implementation trust, but invoking one is not a magic guarantee either: the exported statement, environment, serialization, and checker version must correspond. The simplest first implementation can use a pinned Lean checker in a clean replay process, while documenting the remaining trust. Stronger deployments can add independent checking later.

## 10.4 Hashes identify evidence; they do not create it

A fingerprint is useful for linking a candidate to its request and invalidating stale cache entries. It is not a substitute for a proof or for a well-typed context map. Similarly, an opaque host-language wrapper can make accidental misuse less likely, but its name cannot establish a mathematical proposition. The distinction already made in Leant’s verification receipt is a model to follow [6].

Evidence should be tied to the exact expression that was checked. Adding a wrapper, changing a witness, or rerendering an expression in an environment with different name resolution may require rechecking. A proof that two expressions are equivalent can justify a transport, but the transport is itself part of the evidence. The system must not recycle a favorable status merely because two candidates look similar.

## 10.5 A useful stability property

**Proposition 10.1** (Replay under an unchanged dependency environment). *Let  $\Sigma \vdash p : A$  be a checked judgment. Suppose  $\Sigma'$  is a well-formed extension that preserves all declarations, universe information, and typing rules used in that judgment. Then the same fully resolved term  $p$  has type  $A$  in  $\Sigma'$ .*

*Proof.* Replay the typing derivation. Every constant lookup and every rule application used by the original derivation is still available with the same meaning. Additional unrelated declarations need not be consulted. No source-level name search, instance search, or proof search is required.  $\square$

This limited stability property explains the value of freezing resolved evidence. It does not imply that rerunning an unqualified tactic after adding a new simplification rule will choose the same proof, or that changing a definition used by the theorem preserves anything. Source elaboration and proof discovery are sensitive to their environment in ways that resolved replay need not be.

## 10.6 Repair should be explicit, not silent

When a dependency changes, an artifact should be marked stale according to its actual dependency graph. The system may attempt a repair using the original argument, but should present a semantic difference: changed target, changed hypotheses, changed witness, changed theorem dependencies, or merely changed implementation evidence.

A successful repair of the same approved theorem can be accepted after checking and policy review. A repair that needs stronger hypotheses is a proposed change to the mathematical result. It must not be treated as routine maintenance just because the final file again contains no errors.

# 11 The authoring experience

## 11.1 An interaction centered on missing mathematics

Suppose an author enters the derivative proof of Section 4 but omits openness. The system should first approve the exact theorem statement, then elaborate the induction and stop at the neighborhood obligation. The argument view highlights the differentiation step. The obligation view says that equality on  $s$  has been established but ordinary differentiation at  $x$  requires suitable local equality. The evidence view identifies the attempted derivative-transfer theorem and the missing neighborhood hypothesis.

This response is useful even without a solved theorem. It tells the author which mathematical gap must be addressed. Adding an explicit assumption that  $s$  is open changes the theorem and requires renewed statement approval; proving that  $s$  is a neighborhood from existing assumptions does not. The interface should distinguish these actions rather than present both as ways to make an error disappear.

If the missing work is merely representation management, the experience should be lighter. In the Abel proof, the author can inspect the rational scalar cancellation once, accept the recipe, and leave it collapsed thereafter. A later use should reuse the same certified interface rather than require the author to remember `map_natCast` and the orientation of every rewrite.

## 11.2 Progressive disclosure and an escape hatch

Each contracted step should support expansion from its mathematical sentence to named intermediate facts, then to generated Lean code or proof terms. The author can pin a useful intermediate fact into the argument view. Conversely, a sequence of stable implementation steps can be folded into a recipe without deleting its evidence.

There must be an ordinary Lean escape hatch with the same target and axiom policy. Some proofs depend on library features or constructions for which no suitable recipe exists. Refusing those proofs until the new

language can express everything would make adoption unnecessarily difficult. An escape block should remain part of the source map so that its scope, dependencies, and generated term are visible.

The editor should distinguish a proof that is mathematically unfinished, a proof whose discovery is still running, and a finished proof awaiting replay. It should also show when search solved a step through an unexpectedly strong imported theorem. “Using only” constraints are useful here: they can restrict the visible local premises and the permitted theorem inventory, while keeping the foundational dependencies of those theorems separately auditable.

### 11.3 Learning from repeated successful arguments

A plausible longer-term feature is to suggest reusable recipes from repeated checked patterns. For example, repeated conversion of equality on an open set to derivative equality may justify extracting one helper theorem. Repeated proof-specific partitions may suggest a more general finite-equivalence interface.

The extracted abstraction must be proved and tested like any other library addition. Merely finding similar tactic text is insufficient: the essential domains, side conditions, and type-class assumptions may differ. The system should propose the generalized statement and show how the original uses instantiate it. This turns successful automation into durable mathematical infrastructure rather than an ever-growing store of opaque transcripts.

## 12 An implementation path that does not require a new prover

### 12.1 First build the evidence boundary

The initial prototype should be a Lean-hosted front end, using Lean expressions and existing syntax-extension mechanisms. The first engineering target is not rich prose. It is an exact statement boundary, scoped holes, a small argument syntax, and an export/replay mechanism. A proof that cannot be replayed independently of its search session is a poor foundation for further convenience features.

An initial subset can support typed givens, assumptions, named claims, explicit witnesses, reductions, calculation chains, induction with displayed motives, and Lean escape blocks. Search can initially consist of direct application and a small selection of existing tactics. Every solved step should retain the exact target and evidence, and every incomplete step should remain visibly incomplete.

A crucial invariant is that the target-checking context is not writable by the discovery engine. Provider registration, newly generated helper declarations, and candidate terms must be validated before they enter the approved environment. Otherwise an engine could make a task easy by changing its meaning or assuming its conclusion.

### 12.2 Add the three domain recipes

The next stage should implement only the recipes exercised by this article: local derivative transfer and chain-rule application, rational-algebra/formal-series normalization, and finite equivalence or partition reindexing. Each recipe needs an explicit contract, proof-producing implementation, examples at the weakest intended assumptions, and negative tests.

The recipes should use stable helper lemmas where possible. For example, one derivative-locality theorem can hide the filter bookkeeping; one typed rational-scalar normalizer can expose its equality certificate; and a certified finite equivalence can support many reindexing operations. This minimizes dependence on the precise order of low-level rewrites.

The original inspected theorems provide useful acceptance targets. Their statements should be imported as fixed challenges or compared to independently written equivalents through checked equivalence theorems. The prototype must not simply call the original target theorem and count that as successful reconstruction of its argument. Permitted imports and the candidate inventory need to exclude this trivial leakage.

### 12.3 Integrate Leant behind a narrow adapter

Only after the exact-goal and evidence boundaries work should the prototype connect Leant as an additional reconstruction service. The adapter should first handle closed or cleanly telescope-closed supported goals. Unsupported dependent structure should remain unsupported or opaque for candidate generation, with no unwarranted negative claim.

This route does not require rewriting all of Leant or Djex in Lean. A host-native representation may eventually reduce serialization and context-management friction, but the architecture can start with an external service that returns candidates for exact checking. The important interface is semantic: exact goals, bounded requests, candidate identity, and independently validated results.

The integration should also preserve useful failure information. A structural engine that cannot synthesize an arithmetic lemma has not found a counterexample. A tactic suggestion that closes the current goal may still need inspection for unwanted axioms or dependencies. The document-level acceptance policy is stricter than displaying a plausible suggestion in a REPL.

### 12.4 Add richer discourse only after the core is stable

Natural-language aliases, pronouns, analogy suggestions, and semantic theorem retrieval can be layered on later. Their outputs should resolve to the same typed core. They should not introduce a separate acceptance path in which a fluent explanation bypasses formal contracts.

A practical boundary is to require every mathematical sentence used as an inference to resolve to one of three things: a core proof construct, a registered recipe with a checked contract, or an explicit request to produce a proof of a fixed claim. Explanatory prose outside those forms remains commentary. This keeps the implementation extensible without implying that arbitrary mathematical English has a complete formal semantics.

## 13 How to evaluate whether the language is actually better

### 13.1 Use strong and fair baselines

A convincing experiment should compare four conditions: the inspected source, an expert-refactored Lean version, Lean with the same helper lemmas and automation available to Loom, and the Loom document itself. Comparing only against an unoptimized tactic script would conflate language design with ordinary library improvement.

All conditions should have the same approved theorem statements, foundational policy, dependency versions, and access to mathematical helper results. Any new helper library needed by the proposal should be counted and reported. The original target theorem and equivalent ready-made solutions must not be available as search shortcuts unless the task is explicitly theorem retrieval rather than proof reconstruction.

The three examples here can seed an initial corpus, but they are selected to illustrate the design. A broader evaluation should include unrelated algebra, order theory, combinatorics, analysis, and data constructions,

with both successful and intentionally invalid arguments. Held-out theorem families are needed to distinguish reusable recipes from memorized examples.

### 13.2 Measure authoring, understanding, and replay separately

The most useful measurements concern different parts of the workflow. Authoring measures include expert time, edited tokens, number of interventions, and escape-block use. Comprehension measures can ask readers to identify the invariant, explain a domain restriction, or predict which step fails after a hypothesis is removed. Reconstruction measures include search work, peak memory, success rate, and proof size. Replay measures include checking cost and behavior under controlled dependency changes.

Concision should report both local document size and the amortized cost of new reusable recipes. A shorter proof that requires a large bespoke plugin is not necessarily a success. A small fixed library that shortens many independent proofs may be. The relevant units should therefore include theorem families and maintenance tasks, not only isolated final scripts.

The study should separately report target-fidelity failures, hidden-assumption changes, witness substitutions, and explanation mismatches. A system that rapidly proves the wrong formal interpretation is not outperforming one that exposes an ambiguity. Likewise, a system that returns “unknown” on an invalid step should not be scored as though it had refuted the step, unless it actually supplies the relevant counterexample or impossibility evidence.

### 13.3 Negative tests are part of the language specification

| Test                          | Required behavior                                                                                            |
|-------------------------------|--------------------------------------------------------------------------------------------------------------|
| Pointwise differentiation     | Refuse derivative transfer from equality only at one point; request local equality.                          |
| Nonopen-domain substitution   | Do not silently replace ordinary derivatives by within-set derivatives.                                      |
| Zero scalar denominator       | Expose the missing positive-degree or nonzero condition; do not use field cancellation in an arbitrary ring. |
| Formal-series composition     | Require the relevant substitution evidence; do not infer analytic convergence from a formal identity.        |
| Noninjective reindexing       | Refuse a bijective sum transformation unless multiplicities are explicitly handled by another theorem.       |
| Insufficient induction motive | Show that the hypothesis is unavailable at changed arguments; propose generalization visibly.                |
| Escaping witness              | Reject a witness depending on a variable outside its permitted scope.                                        |
| Leaked branch hypothesis      | Reject use of a fact established only in another case.                                                       |
| Approximate negative search   | Report unknown unless a justified source-level negative conclusion is available.                             |
| Mismatched evidence           | Recheck a changed term or witness; do not transfer a status from another occurrence.                         |
| Unexpected axiom              | Fail the configured axiom policy even when ordinary elaboration succeeds.                                    |
| Changing statement meaning    | Require renewed approval after a domain, binder, instance, or referenced definition changes.                 |

| Test                    | Required behavior                                                       |
|-------------------------|-------------------------------------------------------------------------|
| Unbounded failed stream | Enforce work limits independently of the number of accepted candidates. |

These are proposed acceptance criteria, not results of tests run for this article. Several concern usability as well as logical safety: the desired diagnostic should explain the missing mathematical condition, not merely expose an internal exception.

### 13.4 A falsifiable success criterion

The proposal would be supported if authors can produce and maintain equally strong formal statements with less implementation-specific work, while readers understand the argument at least as well and replay remains reliable. It would be weakened if most savings disappear against well-refactored Lean, if reconstruction routinely needs unpredictable global search, or if hidden semantic choices produce frequent mistakes.

There is no need to promise a universal compression factor. Different subjects and proof styles have different bottlenecks. A useful result could be that a small number of typed mathematical recipes remove substantial friction in analysis and finite combinatorics, while ordinary Lean remains preferable for metaprogramming or unusual dependent constructions. That would still be a meaningful contribution.

## 14 Limits, tradeoffs, and the central recommendation

The principal tradeoff is not rigor versus readability. It is explicit author effort versus the cost and predictability of reconstruction. Hiding more detail can make a document easier to read while making its first elaboration slower or its failures harder to diagnose. Contracts, local search, and evidence expansion are intended to control that tradeoff; they do not abolish it.

Another limit is that mathematical understanding is not exhausted by proving a proposition. An automated engine may find a valid but unenlightening argument. A reader may prefer a proof that reveals a generalization or avoids an unnecessary theorem. The language can preserve the author’s chosen structure and expose dependencies, but no kernel check certifies that an explanation captures the best human insight.

The design also depends on good libraries. A front end cannot indefinitely compensate for missing abstraction boundaries, incompatible normal forms, or poorly scoped lemmas. The ProveIt examples demonstrate both sides: they already contain carefully separated mathematical results, and their remaining friction points suggest reusable interfaces. The most economical implementation may therefore consist of a small proof-document layer plus a growing collection of rigorously specified recipes.

The recommendation is to build Loom as a *contract-based elaboration layer over Lean*, not as a replacement kernel and not as unrestricted English translated by an opaque model. Use Leant’s synthesis strengths to reconstruct scoped logical connections and offer alternatives. Use domain recipes for local analysis, formal-series algebra, and finite transformations. Freeze the statement and the selected constructions, then retain proof evidence that can be independently replayed.

The desired end state is a document in which a mathematician can genuinely write “differentiate the induction identity,” “apply Lagrange inversion,” or “split by membership and reindex,” while a skeptical reader can expand each phrase into every condition that makes it valid. The author should not have to maintain all those conditions as low-level instructions. The verifier should never be asked to take them on trust.

## A A minimal syntax sketch

The following grammar outlines the core, not a completed parser specification. Expressions and binder syntax are delegated to Lean. Mathematical notation and the domain commands used in the case studies would be supplied by registered extensions. Layout, source positions, and error recovery need an implementation-level specification.

```

Document    ::= Declaration*
Declaration ::= "theorem" Name Header "show" Expr
              "proof" Block "end"
              | LeanDeclaration
Header       ::= ("given" Binders | "assume" Name ":" Expr)*
Block        ::= Step*
Step         ::= "let" Name [ ":" Expr ] ":@" Expr
              | "fix" Binders
              | "have" Name ":" Expr "by" Reason
              | "show" Expr "by" Reason
              | "exact" Expr
              | "choose" Name [ ":" Expr ]
                "satisfying" Expr "by" Reason
              | "suffices" Expr "via" Expr "by" Reason
              | Induction | Cases | Calculation | Extension
Reason       ::= Expr | "routine" [Policy] | Block
Induction    ::= "induct" Name ["generalizing" Names]
              CaseBlock+
Cases        ::= "cases" Expr CaseBlock+
Calculation  ::= "calc" (Expr Relation Expr "by" Reason)+
Extension    ::= RegisteredRecipeSyntax

```

Listing 7: Indicative grammar for the proposed typed core.

The parser identifies scope before reconstruction begins. Named facts have stable internal identities. A proof expression is checked against its goal; a domain extension elaborates to ordinary typed obligations and a conditional skeleton. Any unresolved natural-language alias is a diagnostic, not an implicitly added assumption.

## B A recipe contract and its replay record

A locality recipe can be specified abstractly as follows. The displayed ingredients are inputs and obligations, not asserted facts:

```
recipe derivative_from_local_identity
  given f g : Real -> Real
  given s : Set Real, x v : Real
  requires hx      : x in s
  requires hlocal  : s is a neighborhood of x
  requires heq     : forall y in s, f(y) = g(y)
  requires hderiv_g : HasDerivAt g v x
  ensures         : HasDerivAt f v x

implementation
  construct equality_eventually_near_x from hlocal, heq
  transport hderiv_g along that equality
  return the checked proof term
```

Listing 8: Proposed recipe specification for transferring a derivative.

Openness is one way to satisfy `hlocal`; the recipe is intentionally more general than an open-set-only interface. The extra `hx` is useful for the surface context but is not a logical necessity once a suitable neighborhood and equality there are already supplied. A polished library should state the underlying theorem at its minimal assumptions and let the surface recipe manage convenient redundant context.

An artifact for a solved step should have fields corresponding to the following information:

```
StepEvidence {
  source_step_id
  approved_target_and_context
  chosen_witnesses_and_transformation
  resolved_environment_and_dependencies
  proof_term_or_checked_auxiliary_declarations
  recipe_identity_and_version
  exact_request_and_candidate_identity
  replay_toolchain_and_result
  transitive_axiom_inventory
  source_to_obligation_to_evidence_map
}
```

Listing 9: Conceptual evidence record; metadata is not itself a proof.

A checker must validate the proof against the approved target rather than trust a stored `replay_result` field. The same applies to an inventory claimed by an untrusted producer: it should be computed or independently verified. Metadata helps locate and reproduce evidence, while the evidence establishes the typed judgment.

## C Source map and boundaries of this study

The following map identifies the main inspected material. References in the bibliography use immutable repository URLs. The source review was performed through GitHub content retrieval; no local build of these projects was performed.

| Reference | Inspected content and role in the argument                                                                                                                                                             |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| [1]       | Complete module; especially <code>iteratedDeriv_eq_comp</code> and its differentiability wrapper. Grounds the local-calculus example and the insistence on range-only derivative hypotheses.           |
| [2]       | Complete module; especially <code>coeff_exp_subst_of_abel_eq</code> , the full EGF identity, and <code>abelPolynomial_eval_add</code> . Grounds the rational-algebra and generating-function examples. |
| [3]       | Complete module; finite powerset expansion, derivative propagation, and dyadic specialization. Grounds reindexing, generalized induction, domain invariance, and cast distinctions.                    |
| [4]       | Relevant README portions on synthesis, proof suggestions, candidate behavior, and the optional Length layer. Provides user-facing capabilities and limitations, not independent performance evidence.  |
| [5]       | Opening architecture and semantic-origin sections, including candidate authority and provider bindings. Provides the evidence-identity and fragment-boundary precedents.                               |
| [6]       | Complete verification scheduler module. Grounds the distinction between callback acceptance and stronger evidence, accepted-key deduplication, and caller-owned work bounds.                           |

The mathematical results used as examples belong to the cited development; no novelty claim is made for them. The design proposals, conditional architectural arguments, pseudocode, and evaluation plan are the contributions of this article. The relative benefits of the proposed language, the correctness of a future implementation, and compatibility with particular dependency versions remain to be established through implementation and testing.

## References

- [1] Vladimir Reshetnikov et al. *ProveIt: AutonomousIteratedDeriv.lean*. Revision 24ce8bd743eaab64a91ce90725ea00f498d319d2. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AutonomousIteratedDeriv.lean>.
- [2] Vladimir Reshetnikov et al. *ProveIt: AbelPolynomialSeries.lean*. Same ProveIt revision. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AbelPolynomialSeries.lean>.
- [3] Vladimir Reshetnikov et al. *ProveIt: AffineDifferenceOrbit.lean*. Same ProveIt revision. <https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/AffineDifferenceOrbit.lean>.
- [4] Vladimir Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. README. Revision 3a40904be8a410d832d9ab6900b3d3e7b425eccb. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/README.md>.
- [5] Vladimir Reshetnikov et al. *Leant: :synth internals*. Same Leant revision. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/docs/synth-internals.md>.
- [6] Vladimir Reshetnikov et al. *Leant: src/Leant/Synth/Verification.hs*. Same Leant revision. <https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [7] Lean development team. *The Lean Language Reference*. Online reference consulted September 5, 2026; the retrieved reference identified version 4.34.0-rc2. <https://lean-lang.org/doc/reference/latest/>.
- [8] Lean development team. *Elaboration and Compilation*. In the Lean Language Reference. <https://lean-lang.org/doc/reference/latest/Elaboration-and-Compilation/>.
- [9] Lean development team. *Tactic Proofs and Tactic Reference*. In the Lean Language Reference. <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/>; <https://lean-lang.org/doc/reference/latest/Tactic-Proofs/Tactic-Reference/>.
- [10] Lean development team. *Validating a Lean Proof*. In the Lean Language Reference. <https://lean-lang.org/doc/reference/latest/ValidatingProofs/>. Accessed September 5, 2026. Version-sensitive operational details should be rechecked for the deployed toolchain.
- [11] Markus Wenzel and the Isabelle project. *Isar: a Language for Structured Proofs*. Official project introduction. <https://isabelle.in.tum.de/Isar/>.
- [12] Mizar project. *The Mizar Language*. Official language description. <https://mizar.uwb.edu.pl/language/>.
- [13] Naproche project. *Naproche*. Official project introduction and documentation. <https://naproche-net.github.io/>.
- [14] Marcos Cramer et al. *The Naproche Project: Controlled Natural Language Proof Checking of Mathematical Texts*. CEUR Workshop Proceedings, vol. 448, 2009. <https://ceur-ws.org/Vol-448/paper10.pdf>.
- [15] Jannis Limperg and Asta Halkjær From. *Aesop: White-Box Best-First Proof Search for Lean*. Proceedings of CPP 2023, pp. 253–266. <https://doi.org/10.1145/3573105.3575671>.
- [16] Isabelle project. *Sledgehammer*. Official historical description, Isabelle2009 website. Cited for the discovery/reconstruction architecture, not for current feature coverage. <https://isabelle.in.tum.de/website-Isabelle2009/sledgehammer.html>.